Direction Régionale : Casablanca – Settat *Cité des Métiers et des Compétences Nouaceur*

CORRIGÉ OFFICIEL & BARÈME – VARIANTE B (FORMATEUR)

Contrôle Continu N°1 : Moteurs NLP (Année 2026/2027)

Filière : Intelligence Artificielle – Développeur Chatbot
Niveau : Technicien Spécialisé – **Groupe :** DIA_DC_TS_201
Durée : 2H00
Variante : B
Barème : 8 + 6 + 6 = 20

Corrigé Partie I : Questions à Choix Multiples (8 points)

Position des cases justes (de 1re à 4e case) : Q1 : 3e **Q2** : 1re **Q3** : 2e **Q4** : 4e **Q5** : 2e **Q6** : 3e **Q7** : 4e **Q8** : 1re

1. **(1 pt) Histoire du NLP (années 1980) : ■ 3e choix** – *L'apprentissage statistique de modèles probabilistes sur de grands corpus écrits.*
   Justification : Dès les années 1980, les approches basées sur des corpus et des probabilités remplacent les règles manuelles rigides de la 1re génération.

2. **(1 pt) Morphologie (*« introuvables »*) : ■ 1er choix** – *Quatre morphèmes formant les plus petites unités porteuses de sens ou fonction.*
   Justification : `in-` (préfixe de négation), `trouv-` (racine lexicale), `-able` (capacité) et `-s` (marque du pluriel) sont 4 morphèmes.

3. **(1 pt) Architecture NLG vs NLU : ■ 2e choix** – $A = NLG$ et $B = NLU$.
   Justification : Le module **A** transforme une donnée structurée interne en phrase naturelle (**NLG**), puis **B** interprète la réponse écrite du client (**NLU**).

4. **(1 pt) Limite du Stemming : ■ 4e choix** – *Parce qu'il coupe mécaniquement les suffixes sans vérifier le sens réel du mot.*
   Justification : Contrairement à la lemmatisation qui s'appuie sur un lexique, le stemming tronque les mots par règles heuristiques, ce qui peut confondre des familles de mots distinctes.

5. **(1 pt) Lemmatisation spaCy : ■ 2e choix** – `['confirmer', 'commande']`.
   Justification : Le verbe conjugué *« confirmerez »* a pour lemme l'infinitif `'confirmer'` et le nom pluriel *« commandes »* a pour lemme le singulier `'commande'`.

6. **(1 pt) Recherche vectorielle en RAG : ■ 3e choix** – *Retrouver par proximité vectorielle les extraits utiles avant de générer la réponse.*
   Justification : La base vectorielle identifie les passages documentaires les plus proches sémantiquement de la question afin d'alimenter le contexte de génération.

7. **(1 pt) Portes (*gates*) des LSTM et GRU : ■ 4e choix** – *Réguler les données à retenir ou oublier pour garder le contexte sur un texte long.*
   Justification : Les portes contrôlent le flux d'information dans la mémoire récurrente et pallient le problème du *vanishing gradient*.

8. **(1 pt) Cellule LSTM vs GRU : ■ 1er choix** – *La cellule LSTM sépare l'état de cellule et l'état caché via trois portes, augmentant ses paramètres.*
   Justification : Le LSTM possède 3 portes (*input, forget, output*) et deux états (`cell state` + `hidden state`), alors que le GRU fusionne l'état avec 2 portes seulement.

Corrigé Partie II : Web Scraping & Regex (6 points)

Correspondances exactes (1 pt / association) : 1 → F 2 → B 3 → D 4 → E 5 → A 6 → C

N°	Rép.	Justification technique	Instruction Python associée
1	F	Requête HTTP GET vers l'URL cible avec délai max.	`res = requests.get(cible, timeout=5)`
2	B	Recherche BS4 de toutes les balises `<p>` d'une classe.	`res = doc.find_all("p", class_="item")`
3	D	Ciblage Selenium d'un élément unique par son `id`.	`res = driver.find_element(By.ID, "zone")`
4	E	Découpage Regex selon espaces, `;` ou `-` répétés.	`res = re.split(r"[\s;-]+", txt)`
5	A	*Positive Lookbehind* `(?<=DH)` : devise conservée.	`res = re.sub(r"(?<=DH)\s+", "", txt)`
6	C	Substitution non-gourmande (*non-greedy*) `<.+?>`.	`res = re.sub(r"<.+?>", " ", txt)`

Corrigé Partie III : Correction du Code GRU (6 points)

*Barème : **0,5 pt** pour la ligne et l'explication juste + **0,5 pt** pour l'instruction Python corrigée (toute écriture équivalente valide est acceptée, ex. `Dense(len(CATEGORIES), activation='softmax')`).*

N°	Ligne	Explication de l'erreur (0,5 pt)	Instruction Python corrigée (0,5 pt)
1	L. 24	`re.findall()` extrait une liste de motifs et ne remplace pas de texte (`TypeError`). Il faut utiliser `re.sub()`.	`texte = re.sub(r'<.*?>', ' ', texte)`
2	L. 33	La méthode `.transform()` n'existe pas sur le `Tokenizer` de Keras. Il faut utiliser `.texts_to_sequences()`.	`sequences = tokenizer.texts_to_sequences(textes)`
3	L. 34	`pad_sequences()` reçoit `textes` (chaînes brutes) au lieu des séquences d'entiers `sequences`.	`X = pad_sequences(sequences, maxlen=12)`
4	L. 41	Couche de sortie à 2 neurones + `sigmoid`, alors qu'il y a **4 intentions** (`0, 1, 2, 3`) à classer.	`model.add(Dense(4, activation='softmax'))`
5	L. 43	La méthode `model.train()` n'existe pas dans Keras. Il faut appeler `model.fit()`.	`model.fit(X, y, epochs=25, verbose=0)`
6	L. 49	`np.argmin()` sélectionne la probabilité la plus faible au lieu de la plus élevée (`np.argmax()`).	`id_cat = int(np.argmax(probas[0]))`

Annexe Formateur : Script Python Corrigé Complet (Variante B)

Script intégral rectifié et fonctionnel :

```python
import re
import numpy as np
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, GRU, Dense

# 1. Donnees d'entrainement du Chatbot bancaire (4 intentions : 0, 1, 2, 3)
CATEGORIES = {0: "solde", 1: "carte_perdue", 2: "virement", 3: "reclamation"}
requetes_brutes = [
    "<div>Quel est le montant disponible sur mon compte ?</div>",
    "<p>J'ai perdu ma carte bancaire ce matin !</p>",
    "Je souhaite effectuer un virement vers un autre compte",
    "<span>Frais bancaires preleves par erreur sur mon releve</span>",
    "Bonjour, comment consulter mon solde actuel svp ?",
    "<b>Urgent</b> bloquer ma carte volee immediatement",
    "Envoyer de l'argent a un beneficiaire au Maroc",
    "Contester une operation anormale sur ma facture"
]
labels = [0, 1, 2, 3, 0, 1, 2, 3]

# 2. Nettoyage du texte avec Regex
def nettoyer(texte):
    texte = re.sub(r'<.*?>', ' ', texte)           # CORR 1 : re.sub()
    texte = texte.lower()
    return re.sub(r'\s+', ' ', texte).strip()

textes = [nettoyer(r) for r in requetes_brutes]

# 3. Tokenisation et Padding
tokenizer = Tokenizer(num_words=150)
tokenizer.fit_on_texts(textes)
sequences = tokenizer.texts_to_sequences(textes)  # CORR 2 : texts_to_sequences()
X = pad_sequences(sequences, maxlen=12)           # CORR 3 : sequences (entiers)
y = np.array(labels)

# 4. Modele GRU pour classifier les 4 intentions
model = Sequential()
model.add(Embedding(input_dim=150, output_dim=16, input_length=12))
model.add(GRU(16))
model.add(Dense(4, activation='softmax'))         # CORR 4 : 4 classes + softmax
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.fit(X, y, epochs=25, verbose=0)             # CORR 5 : model.fit()

# 5. Test sur un nouveau message client
seq_test = tokenizer.texts_to_sequences([nettoyer("Ma carte est perdue")])
X_test = pad_sequences(seq_test, maxlen=12)
probas = model.predict(X_test)
id_cat = int(np.argmax(probas[0]))                # CORR 6 : np.argmax()
print("Intention predite :", CATEGORIES[id_cat])
```